COMPILER CONSTRUCTION

Lexical Analysis Using JFlex

Third-Year Computer Science

Question: Designing a Lexical Analyzer for an Ugali Preparation Language

Consider a small domain-specific language, Ugali Preparation Language (UPL), used to describe the procedure for preparing ugali. A valid UPL program may contain instructions such as:

```
prepare ingredients;
boil 4 cups water;
add 2 cups flour;
stir;
press ugali;
turn ugali;
cook 8 minutes;
check consistency;
shape ugali;
serve ugali;
```

The following Context-Free Grammar describes part of the language:

```
S -> Preparation Boiling Mixing Cooking Checking Shaping Serving

Preparation -> PREPARE INGREDIENTS SEMICOLON
Boiling -> BOIL Quantity WATER SEMICOLON
Mixing -> ADD Quantity FLOUR SEMICOLON STIR SEMICOLON Mixing
Mixing -> ADD Quantity FLOUR SEMICOLON STIR SEMICOLON
Cooking -> PRESS UGALI SEMICOLON TURN UGALI SEMICOLON Cooking
Cooking -> COOK Duration SEMICOLON
Checking -> CHECK CONSISTENCY SEMICOLON
Shaping -> SHAPE UGALI SEMICOLON
Serving -> SERVE UGALI SEMICOLON
Quantity -> NUMBER CUPS
Duration -> NUMBER MINUTES
```

a) Lexical specification - 6 marks

Identify and classify the lexemes in the language into appropriate token categories. Your scanner should recognize at least the following reserved words:

```
prepare, ingredients, boil, water, add, flour, stir, press, turn, cook, check,
consistency, shape, serve, ugali, cups, minutes
```

It should also recognize integer literals, semicolon (;), comments, and whitespace. Give suitable token names for each category.

b) Regular expressions - 8 marks

Write appropriate regular expressions for recognizing: integer literals, identifiers, whitespace, single-line comments beginning with //, reserved words, and the semicolon delimiter.

Identifiers should follow the rule:

```
letter (letter | digit | _)*
```

Examples of valid identifiers include:

```
temperature   maize_flour   step2   cooking_time
```

c) JFlex implementation - 14 marks

Write a complete JFlex specification named UgaliLexer.flex. The scanner should:

- Recognize all reserved words and return their corresponding tokens.
- Recognize integer literals and preserve their numerical value.
- Recognize identifiers.

- Recognize semicolons.
- Ignore spaces, tabs and line breaks.
- Ignore single-line comments.
- Maintain the current line and column number.
- Detect illegal characters.
- Display a meaningful lexical error containing the offending character, line number and column number.

Example:

```
Lexical error at line 4, column 7: illegal character '@'
```

d) Token attributes - 6 marks

Modify your scanner so that when an integer is encountered, both its token type and value are available to later phases of the compiler.

For the input:

```
boil 4 cups water;
```

The lexical analyzer should conceptually generate:

```
BOIL
NUMBER(4)
CUPS
WATER
SEMICOLON
```

Explain why storing the value 4 is important for later compiler phases.

e) Reserved words versus identifiers - 6 marks

Suppose the following input is supplied:

```
cook cooking_time;
serve server;
```

Explain how the scanner distinguishes cook from cooking_time, and serve from server. Discuss how the longest-match rule and rule ordering in JFlex affect this process.

f) Test the scanner - 8 marks

Use your scanner to analyze the following valid input:

```
// Ugali preparation program

prepare ingredients;
boil 4 cups water;
add 2 cups flour;
stir;
add 1 cups flour;
stir;
press ugali;
turn ugali;
cook 8 minutes;
check consistency;
shape ugali;
serve ugali;
```

Write the token stream that should be produced. Use a form similar to:

```
PREPARE
INGREDIENTS
SEMICOLON
BOIL
NUMBER(4)
CUPS
WATER
SEMICOLON
...
```

g) Lexical error detection - 7 marks

Consider the following invalid input:

```
prepare ingredients;
boil 4 cups water;
add @2 cups flour;
cook 5# minutes;
```

Identify all characters that should result in lexical errors. Show the error messages your scanner should generate, assuming accurate line and column tracking.

h) Extension: lexical states - 10 marks

Extend the language to support multiline comments of the form:

```
/*
   Ugali should be stirred continuously.
   Add flour gradually.
*/
```

Using JFlex lexical states, design a scanner mechanism that enters a comment state after /* and returns to the normal scanning state after */. Your solution should also detect an unterminated multiline comment.

i) Scanner and parser relationship - 10 marks

Explain how the JFlex-generated lexical analyzer could be integrated with a parser generated using a tool such as CUP. Your answer should discuss token generation, passing tokens from the scanner to the parser, token attributes, the role of the symbol table, and lexical errors versus syntax errors.

j) Analysis question - 10 marks

Consider the input:

```
boil four cups water;
```

Assume that four matches the identifier regular expression. Explain: (1) which tokens the lexical analyzer would generate; (2) whether the lexical analyzer should report an error; (3) which compiler phase should detect that the statement does not conform to Quantity -> NUMBER CUPS; and (4) why this illustrates the separation between lexical analysis and syntax analysis.

Total: 85 Marks

Learning Outcomes

- Design regular expressions for a programming language.
- Implement a scanner using JFlex.
- Distinguish reserved words from identifiers.
- Handle token attributes.
- Implement lexical error reporting.
- Apply longest-match and rule-priority principles.
- Use lexical states.
- Relate lexical analysis to syntax analysis.
- Explain how JFlex can be integrated with a parser generator.